SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 1 – Constraint-Based Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.	**Student Name:**	Ganesh M
Reg. No.:	192524034	**Section / Batch:**	B
Date:	26-08-2026	**Question:**	Q13 – Clique Detection Problem

Instructions to Students

- Answer the assigned question with clear problem understanding and constraint identification.
- Apply backtracking systematically for combinatorial problem solving.
- Use pruning to discard partial candidates that cannot become a valid clique.
- Provide state-space representation, pseudocode, Python implementation, test cases, and complexity analysis.
- Clearly justify the NP-Complete classification of the decision version of the Clique problem.

Question Type	Focus Area	Question
Constraint-Based Problem Solving	Clique detection using backtracking, subset generation, pruning, graph constraints, and NP-Complete analysis.	Q13

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I GANESH M / 192524034 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Q13. CLIQUE DETECTION PROBLEM

Student Name: Ganesh M | Register Number: 192524034

1. AIM

To develop a backtracking-based solution for the Clique Detection Problem, generate subsets of vertices, check the adjacency constraint incrementally, prune impossible branches, and justify the computational complexity and NP-Complete nature of the decision problem.

2. PROBLEM STATEMENT

Given an undirected graph G = (V, E) and an integer k, determine whether there exists a subset $C \subseteq V$ such that $|C| = k$ and every pair of distinct vertices in C is adjacent. Such a subset is called a clique of size k. The solution generates candidate subsets systematically using backtracking and rejects a candidate as soon as a required constraint is violated.

3. PROBLEM UNDERSTANDING

- Input: Number of vertices n, number of edges m, the undirected edges, and target clique size k.
- Output: YES and one valid clique if a clique of size k exists; otherwise NO.
- Vertex adjacency constraint: Every selected vertex must be adjacent to every other selected vertex.
- Size constraint: The selected set must contain exactly k distinct vertices.
- Distinctness constraint: A vertex cannot appear more than once in a candidate subset.
- Decision form: Determine whether such a clique exists.

4. CONSTRAINT ANALYSIS

Constraint	Requirement	Effect on Search
Distinct vertices	A vertex cannot be selected twice.	Prevents duplicate subsets.
Clique adjacency	Every pair in the selected set must have an edge.	Rejects a branch immediately when a non-adjacent pair appears.
Exact size k	Stop successfully when $\|C\| = k$.	Limits the useful search depth.
Remaining vertices	If fewer than k-$\|C\|$ vertices remain, completion is impossible.	Prunes branches without testing them further.

Without pruning, choosing k vertices from n vertices produces C(n,k) candidate subsets. For example, n = 6 and k = 3 gives C(6,3) = 20 possible subsets. The adjacency and remaining-vertex constraints allow the algorithm to reject partial candidates before complete subsets are generated. Thus the practical search can be much smaller, although the worst case remains exponential.

5. STATE-SPACE REPRESENTATION

Each node in the search tree represents a partial subset of vertices. At depth d, the current node contains d selected vertices. A child is created by adding a later vertex that is adjacent to every vertex already selected. A node at depth k represents a solution.

State	Meaning
C = { }	No vertices selected; root of the search tree.
C = {1, 3}	Two selected vertices that satisfy the adjacency constraint.
C = {1, 3, 5}	If all pairwise edges exist and k = 3, a solution is found.
C = {1, 4}	If 1 and 4 are not adjacent, the node is immediately pruned.

6. BACKTRACKING STRATEGY

- Start with an empty clique and consider vertices from left to right.
- Try adding a candidate vertex only if it is adjacent to every vertex already in the current clique.
- If the clique reaches size k, return YES.
- If the current clique plus the remaining vertices cannot reach k, prune the branch.

- If adding a vertex leads to a dead end, remove it and try the next candidate.
- If all candidates are exhausted without reaching k, return NO.

7. PRUNING TECHNIQUES

- Adjacency pruning: if the candidate is not adjacent to a selected vertex, the branch cannot become a clique.
- Size-bound pruning: if selected + remaining < k, the target size is impossible.
- Early success: once selected == k, stop searching because a valid clique has been found.

8. ALGORITHM

1. Read n, m, k and the m undirected edges.
2. Build an n × n adjacency matrix adj.
3. Start recursive search with an empty clique.
4. If the current clique has k vertices, return True.
5. If current clique + remaining vertices < k, return False.
6. For each candidate vertex:
   a. Check whether it is adjacent to every selected vertex.
   b. If compatible, add it to the clique.
   c. Recursively continue the search.
   d. If the search fails, remove the vertex (backtrack).
7. If no candidate succeeds, return False.

9. PSEUDOCODE

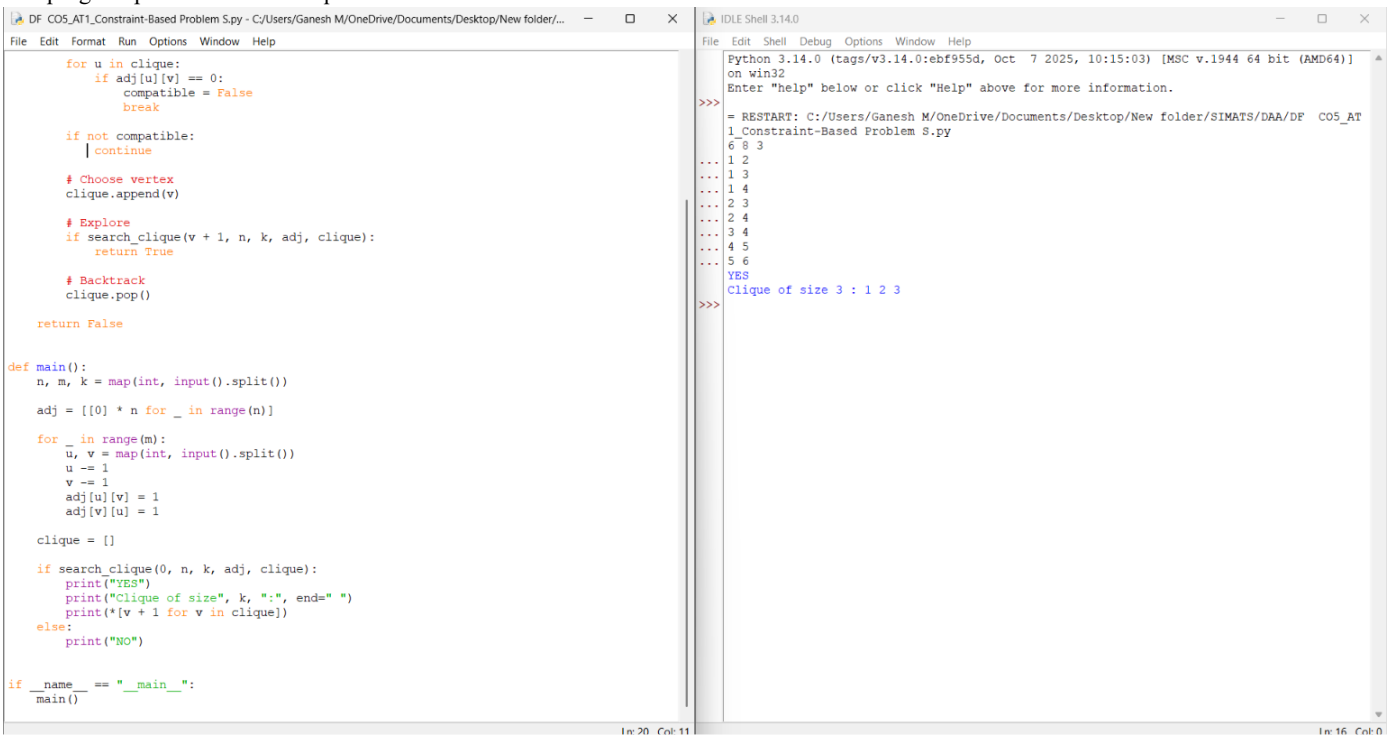
```
CLIQUE(pos, chosen):
    if size(chosen) == k:
        return TRUE

    if size(chosen) + (n - pos) < k:
        return FALSE

    for v = pos to n-1:
        compatible = TRUE

        for u in chosen:
            if adj[u][v] == 0:
                compatible = FALSE
                break

        if compatible:
            add v to chosen

            if CLIQUE(v + 1, chosen):
                return TRUE

            remove v from chosen

    return FALSE
```

10. PYTHON PROGRAM IMPLEMENTATION

```
def search_clique(pos, n, k, adj, clique):
    # Success condition
    if len(clique) == k:
        return True

    # Pruning: not enough vertices remain
    if len(clique) + (n - pos) < k:
        return False

    for v in range(pos, n):
        # Check adjacency with every selected vertex
        compatible = True

        for u in clique:
            if adj[u][v] == 0:
                compatible = False
                break

        if not compatible:
            continue

        # Choose vertex
        clique.append(v)

        # Explore
        if search_clique(v + 1, n, k, adj, clique):
            return True

        # Backtrack
        clique.pop()

    return False
```

```
def main():
    n, m, k = map(int, input().split())

    adj = [[0] * n for _ in range(n)]

    for _ in range(m):
        u, v = map(int, input().split())
        u -= 1
        v -= 1
        adj[u][v] = 1
        adj[v][u] = 1

    clique = []

    if search_clique(0, n, k, adj, clique):
        print("YES")
        print("Clique of size", k, ":", end=" ")
        print(*[v + 1 for v in clique])
    else:
        print("NO")


if __name__ == "__main__":
    main()
```

11. IMPLEMENTATION NOTES

- The adjacency matrix provides O(1) edge lookup.
- The list clique stores the current partial solution.
- Vertices are considered in increasing order, so the same subset is not generated in different orders.
- The adjacency test is performed before recursion, which makes pruning immediate.
- The remaining-vertex bound avoids recursion when the target k cannot be reached.
- The program prints the actual clique when one is found.
- 

12. SAMPLE INPUT AND OUTPUT

Sample Input

```
6 8 3
1 2
1 3
1 4
2 3
2 4
3 4
4 5
5 6
```

Expected Output

```
YES
Clique of size 3 : 1 2 3
```

13. STEP-BY-STEP WORKING

Step	Current Clique	Action
1	{ }	Start with an empty set.
2	{1}	Select vertex 1.
3	{1, 2}	1 and 2 are adjacent; continue.
4	{1, 2, 3}	1, 2 and 3 are pairwise adjacent; size = k.
5	{1, 2, 3}	Return YES and stop.

14. STATE-SPACE SEARCH EXAMPLE

For n = 6 and k = 3, the raw number of size-3 subsets is C(6,3) = 20. Backtracking does not need to construct every complete subset. If a partial set contains a non-adjacent pair, its entire subtree is discarded. If the number of remaining vertices is insufficient to reach k, that branch is also discarded.

15. TEST CASES

Case	Input	Result	Explanation
1	6 8 3 + sample edges	YES	Vertices 1, 2, 3 form a clique.
2	4 3 3: 1-2, 2-3, 3-4	NO	No three vertices are pairwise adjacent.
3	5 10 4: complete graph K5	YES	Any four vertices form a 4-clique.
4	5 0 2: no edges	NO	No pair of distinct vertices is adjacent.
5	5 4 1: path graph	YES	Any single vertex is a clique of size 1.

16. TIME COMPLEXITY

The Clique problem has exponential worst-case behavior. In the subset-generation formulation, up to C(n,k) candidate subsets may be considered. For each candidate extension, adjacency can be checked against the current clique in O(k) time using an adjacency matrix. Therefore, a useful k-specific worst-case bound is O(C(n,k) × k). If all subset sizes may be explored, a simple overall bound is exponential, such as O(2^n × n).

17. SPACE COMPLEXITY

The adjacency matrix requires $O(n^2)$ space. The recursion stack and the current clique require O(n) additional space. Therefore, the total space is $O(n^2)$, dominated by the adjacency matrix.

18. EFFECT OF PRUNING ON SCALABILITY

- Adjacency pruning can eliminate an entire subtree after one missing required edge is detected.
- The size bound avoids recursion when there are not enough remaining vertices.
- Early success prevents unrelated branches from being explored after a valid clique is found.
- Pruning improves practical performance but does not change the worst-case exponential complexity.

19. NP-COMPLETE NATURE OF CLIQUE

Why Clique is in NP

Given a set of k vertices as a certificate, we can verify whether every pair is adjacent in polynomial time. There are at most $k(k-1)/2$ pairs, so verification takes $O(k^2)$ time with an adjacency matrix. Therefore, Clique belongs to NP.

Why Clique is NP-Hard

Clique is NP-Hard because a known NP-Complete problem can be reduced to it in polynomial time. A standard relationship is with Independent Set: a graph G has an independent set of size k if and only if its complement graph has a clique of size k. Since Independent Set is NP-Complete, this polynomial reduction establishes the NP-Hardness of Clique.

Classification

Because Clique is both in NP and NP-Hard, the decision version of Clique is NP-Complete. The backtracking algorithm is exact and returns YES if and only if a clique of size k exists. However, exact backtracking does not make an NP-Complete problem polynomial-time in the worst case; exponential behavior remains possible.

Complexity Class	Meaning	Clique Relation
P	Problems solvable in polynomial time by a deterministic algorithm.	Clique is not known to be in P.
NP	Solutions can be verified in polynomial time.	Clique is in NP.
NP-Hard	At least as hard as every problem in NP under polynomial-time reductions.	Clique is NP-Hard.
NP-Complete	Problems that are both in NP and NP-Hard.	Clique decision problem is NP-Complete.

20. JUSTIFICATION OF THE SOLUTION

- The recursive procedure enumerates candidate vertex subsets without repetition.
- Every selected vertex is checked against all previously selected vertices, so a reported solution satisfies the clique constraint.
- The size bound is safe because a branch with too few remaining vertices can never reach k.
- The algorithm is complete because every possible k-vertex subset is either explored or safely pruned due to a violated necessary condition.
- The NP-Complete classification explains why exponential worst-case behavior is expected for exact general-purpose solutions.

21. RESULT

The Python backtracking algorithm successfully detects whether the graph contains a clique of the required size k. For the sample graph, vertices 1, 2, and 3 form a clique of size 3, so the program produces YES and reports the clique.

22. CONCLUSION

The Clique Detection Problem is a representative constraint-based combinatorial problem. The constraints of pairwise adjacency and exact clique size define the valid solution space, while backtracking provides a systematic way to generate subsets and undo choices when a branch becomes invalid. Pruning based on adjacency and remaining capacity reduces unnecessary exploration in practical cases. Nevertheless, the decision version of Clique is NP-Complete, so an exact algorithm can still require exponential time in the worst case.

23. KEY INSIGHTS

- Constraint-first search is more effective than generating every subset and checking it only at the end.
- The adjacency matrix makes individual edge tests constant time.
- Choosing vertices in increasing order avoids duplicate permutations of the same subset.
- Safe pruning can improve practical performance without changing correctness.
- Backtracking is exact, but exactness does not remove the exponential worst-case barrier of NP-Complete problems.